

1. 소개 및 개발 환경 구축

1. OpenAI 계정 생성 및 크레딧 확인 방법

OpenAI API를 사용하려면 먼저 OpenAI 계정을 만들어야 합니다. 이미 ChatGPT 웹 서비스를 사용 중이더라도, OpenAI 개발자 플랫폼 계정이 별도로 필요합니다.

- **계정 생성:** OpenAI 플랫폼 웹사이트에 접속하여 이메일로 회원가입을 진행합니다. 이메일 인증 및 전화번호 인증 단계를 거쳐 계정을 활성화합니다. 기존에 ChatGPT를 사용한 적이 있다면 해당 이메일로 로그인하면 됩니다.
- **크레딧 확인:** 예전 OpenAI는 신규 가입자에게 일정 금액의 API 무료 크레딧을 제공했지만, **2024년 이후로는 신규 계정에 무료 크레딧을 제공하지 않습니다.** 계정 생성 후 **Dashboard → Usage** 페이지에서 현재 사용할 수 있는 크레딧이나 사용량을 확인할 수 있습니다. API를 계속 사용하기 위해 결제 수단을 등록하고 크레딧(최소 \$5)을 충전해야 합니다. (OpenAI API는 선결제한 크레딧이나 월말 후불 결제를 통해 **Pay-as-you-go** 방식으로 요금이 청구됩니다.)

##실습1.1. OpenAI 로그인 및 크레딧 확인

OpenAI 계정 생성, 결제 정보 등록 및 Credit balance 추가하기

- **사용량 대시보드:** OpenAI 플랫폼의 **Usage** 대시보드에서는 월별 사용량과 비용을 확인할 수 있습니다. 실습 중에는 응답에 사용된 **토큰(token)** 수를 확인하여 예산을 관리하는 것이 좋습니다. (**Usage** 페이지에서는 **일일 사용량과 크레딧 소진 현황** 등을 볼 수 있습니다.)

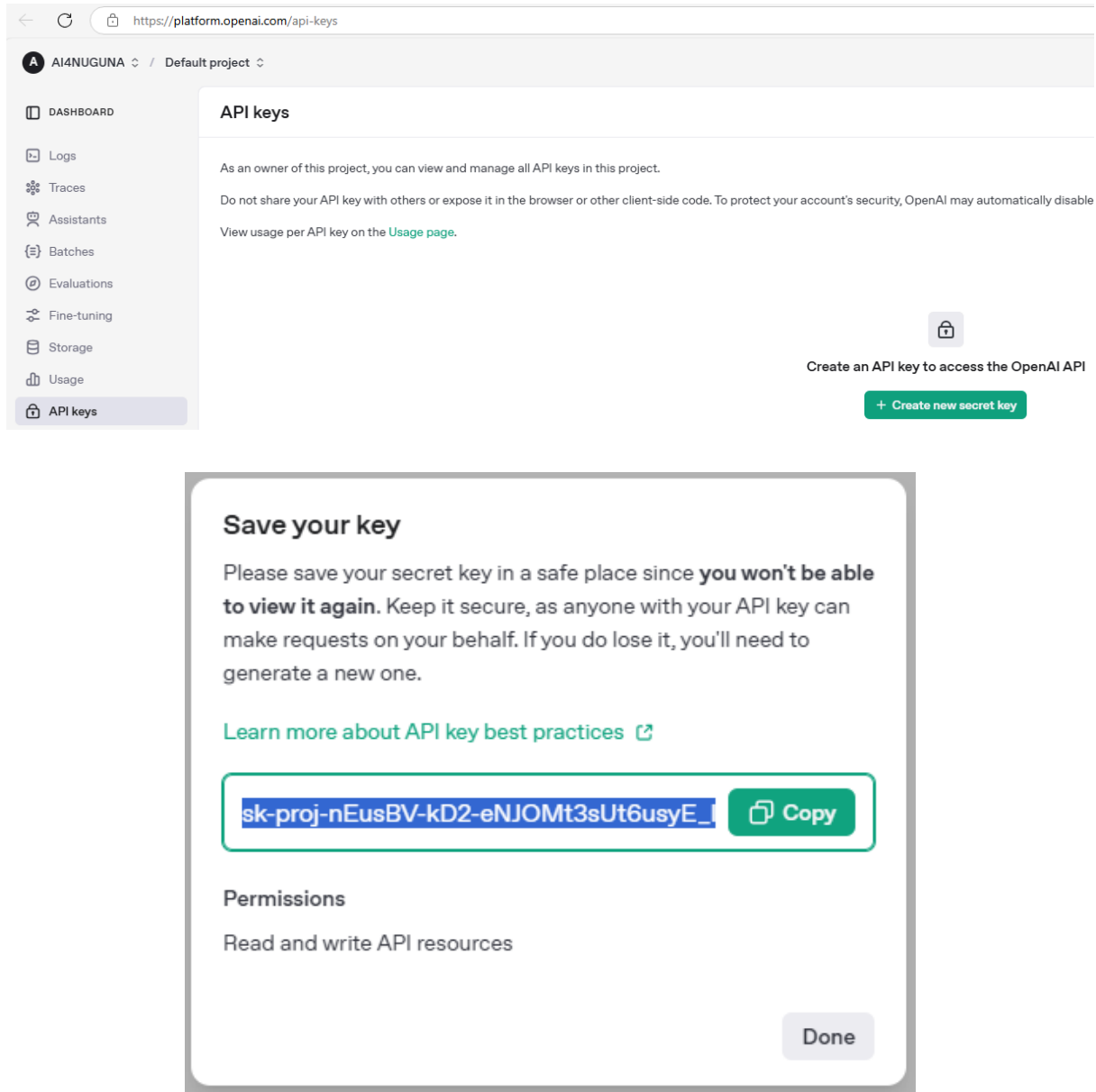
 참고: OpenAI API와 ChatGPT Plus는 다른 서비스입니다. ChatGPT Plus를 구독해도 별도의 API 크레딧이 제공되지 않으며, API를 사용하려면 반드시 위와 같이 별도 결제 또는 크레딧 충전이 필요합니다.

2. OpenAI API Key 발급 및 관리 방법

OpenAI 계정을 만들었으면, 이제 API를 호출하기 위한 API 키(secret key)를 발급받아야 합니다. API 키는 개발자용 개인 비밀번호와 같아서, 각 요청을 인증하는 데 사용되며 **절대로 다른 사람과 공유하거나 공개 저장소에 노출되어서는 안 됩니다.** 발급 및 관리 방법은 다음과 같습니다:

##실습1.2. API 키 발급

- **API 키 발급:** OpenAI 플랫폼에서 우측 상단의 프로필 메뉴를 클릭하고 **"View API Keys"** (API 키 보기) 섹션으로 이동합니다. 페이지에서 **"Create new secret key"** 버튼을 눌러 새로운 API 키를 생성하세요. 키에 대해 이름을 지정할 수 있는데, 사용 용도에 따라 **"ColabTestKey"** 등으로 구분해두면 추후 관리에 편리합니다.



- **키 복사 및 저장:** 새 API 키가 생성되면 **한 번만** 전체 키 문자열이 화면에 표시됩니다. 이 때 **키를 안전한 곳에 복사**해 두세요(예: 메모장에 임시 저장 후 나중에 안전한 비밀 관리 서비스에 보관). 한 번 창을 닫고 나면 OpenAI 플랫폼에서는 같은 키를 다시는 전문 (full key)을 보여주지 않습니다. 키의 형식은 **sk-...**로 시작하는 약 51자 길이의 문자열입니다.
- **API 키 관리:** 필요에 따라 여러 개의 API 키를 발급할 수도 있습니다. 예를 들어 프로젝트별로 다른 키를 쓰거나 팀별로 키를 나눠 사용할 수 있습니다. 키 목록에서 각 키의 마

지막 사용 날짜를 확인하거나 키를 삭제(revoke)할 수 있으므로, **분기별로 키를 교체**하거나 유출 위험이 있는 키는 즉시 폐기하는 등 보안 관리에 유의하세요.

 **베스트 프랙티스:** 팀 프로젝트의 경우 팀원마다 각자의 API 키를 발급해 사용하는 것이 권장됩니다. 또한 공개 저장소(github 등)에 키가 실수로 노출되지 않도록 .env 파일이나 환경 변수로 관리하고, 노출 시 즉시 키를 폐기하십시오.

3. Google Colab 환경 설정

이제 실습을 위해 Google Colab 환경을 설정합니다. **Google Colab**은 브라우저에서 실행되는 무료 **Jupyter 노트북 환경**으로, 별도의 로컬 세팅 없이 Python 코드를 실행할 수 있어 편리합니다. 이미 Colab 사용 경험이 있다고 가정하지만, 원활한 실습을 위해 몇 가지 사항을 점검하겠습니다.

- **Colab 노트북 열기:** Chrome 등 웹 브라우저에서 Google Colab에 접속하여 새로운 Python 3 노트북을 생성합니다. (Google 계정으로 로그인 필요)
- **런타임 연결:** 상단의 "Connect" 버튼을 눌러 Colab 런타임을 활성화하세요. 최초 실행 시에는 기본적으로 CPU 런타임이 연결됩니다. OpenAI API 실습에는 CPU로도 문제 없으며, GPU/TPU 가속은 필요없습니다.
- **Python 버전 확인:** Colab의 기본 Python 버전은 3.10 이상으로 OpenAI SDK 요구 사항(>=3.8)을 충족합니다. 아래와 같이 Python 버전을 확인할 수 있습니다.

```
!python --version
```

```
Python 3.11.13
```

- **인터넷 액세스:** Colab은 인터넷에 접속할 수 있으므로, OpenAI API 호출이 가능합니다. 다만 매우 드문 경우이지만 네트워크 문제가 발생하면 런타임을 재시작하거나 인터넷 연결을 점검해야 합니다.
- **Google 계정 드라이브 연동 (선택):** 필요시 자신의 구글 드라이브를 마운트하여 노트북과 데이터를 주고받을 수 있습니다. 하지만 이번 세션에서는 주로 API 호출 위주이므로 드라이브 연동은 필수는 아닙니다.

 **유지 사항:** Colab의 세션은 일정 시간이 지나면 초기화됩니다. 노트북을 종료했다 다시 열면 이전에 설치한 패키지나 설정한 환경변수

등이 사라지므로, 노트북을 열 때마다 아래 소개하는 초기 설정 코드를 다시 실행해야 한다는 점을 기억하세요.

4. Google Colab에서 API 키 설정

OpenAI API 키는 민감 정보이므로, 코드에 직접 하드코딩하지 않고 **Colab의 보안 기능**을 이용해 관리합니다. 2023년 말부터 Colab에는 **"Secrets" (사용자 비밀)** 기능이 도입되어 노트북 내에서 API 키를 안전하게 저장하고 불러올 수 있습니다. 다음은 Colab에서 API 키를 등록하는 방법입니다.

1. **시크릿 추가:** Colab 노트북의 왼쪽 사이드바에서 열쇠 모양  아이콘 (Secrets)을 클릭합니다. **"Add a new secret"** 버튼을 누르고, Name에 `OPENAI_API_KEY` (대문자)로 입력하고, Value에 앞서 발급받은 API 키 값을 붙여넣습니다. **주의:** Secret 이름은 한번 정하면 변경할 수 없으니 정확히 입력하세요.
2. **노트북 액세스 설정:** 방금 추가한 `OPENAI_API_KEY` secret의 우측에 토글 스위치가 있습니다. 이를 활성화하면 현재 노트북에서 해당 Secret 값을 사용할 수 있습니다. (기본적으로 추가 직후 활성화되어 있을 것입니다.)
3. **코드에서 키 불러오기:** 이제 Python 코드에서 Colab Secrets에 저장된 키를 불러와 환경 변수로 등록합니다. 아래 코드를 노트북에 입력하고 실행하세요:

##실습1.3. Colab에 API Key 등록

```
from google.colab import userdata
import os

# Colab에 저장한 Secret에서 API 키 가져와 환경 변수에 등록
os.environ["OPENAI_API_KEY"] = userdata.get('OPENAI_API_KEY')

# 제대로 설정됐는지 확인 (키 값은 출력하지 않도록 주의)
print("API Key configured:", "OPENAI_API_KEY" in os.environ)
```

첫 실행 시 아무 출력도 나타나지 않으면 **정상적으로 완료**된 것입니다 (`"API Key configured: True"` 등이 출력될 수 있음). 이렇게 하면 현재 Colab 세션의 환경변수 `OPENAI_API_KEY` 에 우리가 입력한 키가 설정됩니다. OpenAI Python SDK는 기본적으로 이 환경변수를 사용하여 인증을 수행하므로, 이후 코드에서 별도로 키를 하드코딩할 필요가 없습니다.

- **Secrets 활용 장점:** 키 값을 코드에 노출하지 않으므로 GitHub 등에 노트북을 공유하더라도 안전합니다. 또한 Colab의 Secrets는 본인에게만 보이며, 노트북별로 접근 제어가 가능하여 다른 노트북이나 사용자와 공유되지 않습니다.

- **잠재적 오류:** `userdata.get('OPENAI_API_KEY')` 를 호출했는데 *키가 없거나 None* 이 반환된다면, 앞서 Secrets에 제대로 추가되었는지 또는 노트북 액세스가 켜져 있는지 확인하세요. 또한 `os.environ` 에 설정하는 코드를 빼먹으면 OpenAI SDK에서 키를 못 찾게 되니 반드시 `os.environ[...] = ...` 할당을 포함해야 합니다.
- **주의:** `userdata.get('OPENAI_API_KEY')` 호출 자체는 키 값을 반환하기 때문에, **이 값을 직접 출력하거나 사용하지 않도록 주의**하세요. 위 코드처럼 바로 환경변수에 넣거나 변수에 저장만 하고, 노트북 화면에 노출하지 않는 것이 좋습니다.

✅ **확인:** 위 코드를 실행한 후 `os.environ["OPENAI_API_KEY"]`로 키가 잘 설정되었는지 확인해보세요. 만약 설정이 안 되었거나 오타가 있다면, 이후 단계에서 인증 오류가 발생하니 이 단계에서 정확히 설정하도록 합니다.

5. OpenAI Python SDK (v1.xx 이상) 설치 및 설정

OpenAI의 Python용 SDK 라이브러리(`openai` 패키지)를 설치하고 사용할 준비를 합니다. Colab 노트북에 기본적으로 `openai` 라이브러리가 설치되어 있지 않으므로 `pip` 을 통해 설치해야 합니다(혹시 이전에 설치했다면 업그레이드). 버전 1.xx 이상의 최신버전을 사용하는 것이 중요합니다. 버전 1.xx에서는 API 호출 방식이 이전 0.xx 버전과 조금 다르므로, 구버전을 사용할 경우 혼란이나 오류가 발생할 수 있습니다.

##실습1.4. OpenAI Library 설치 및 버전 확인

- **설치:** 아래 명령을 Colab 셀에 실행하여 OpenAI Python 라이브러리를 설치합니다.
U 옵션은 최신 버전으로 업그레이드하라는 의미입니다.

```
!pip install -U openai
```

설치가 완료되면, 라이브러리를 불러와 버전을 확인해봅시다:

```
import openai
print("OpenAI Library Version:", openai.__version__)
```

예상 출력 (버전은 예시):

```
OpenAI Library Version: 1.99.9
```

이와 같이 1.xx 이상의 버전이 설치되었는지 확인합니다. 만약 여전히 0.xx 버전이 나온다면, 런타임을 재시작한 뒤 다시 설치를 시도하세요. Colab에서 라이브러리를 업그레이드한 후 즉시 사용하려 할 때 충돌이 발생하면, 메뉴에서 "Runtime → Restart runtime"을 해준 뒤 `import openai` 를 다시 수행하면 됩니다.

- **SDK 설정:** 최신 버전의 OpenAI SDK에서는 초기 설정이 간소화되었습니다. 과거에는 `openai.api_key = "sk-..."` 처럼 키를 파이썬 코드에서 설정했지만, 이제는 이전 단계에서 설정한 **환경 변수**를 자동으로 활용합니다. 즉, `openai` 라이브러리를 통해 **Client** 객체를 생성할 때 명시적으로 키를 주지 않아도 `OPENAI_API_KEY` 환경변수를 읽어들이 인증을 수행합니다.
- **추가 설정:** 조직 ID 등이 있다면 `openai.organization = "...ID..."` 식으로 지정할 수도 있지만, 일반 개인 계정의 경우 기본 조직이 자동 적용되므로 신경 쓰지 않아도 됩니다. 기본 요청 타임아웃이나 프록시 등이 필요한 경우 `OpenAI(api_key=..., timeout=..., api_base=...)` 와 같은 파라미터를 사용할 수 있습니다.

💡 Note: OpenAI SDK v1.xx와 v0.xx의 사용법 차이를 인지하세요. 예를 들어, **ChatCompletion API**를 호출할 때 0.xx 버전에서는 `openai.ChatCompletion.create()`를 사용했지만 1.xx 버전에서는 **client 객체의 메서드를 사용**합니다. 이 부분은 다음 단계에서 실습으로 다룰 것입니다. 만약 문서나 예제 코드를 검색할 때 버전이 다른 코드가 보이면, 현재 권장 방식으로 변환해서 사용해야 함을 기억하세요.

6. OpenAI 클라이언트 인스턴스 생성 (`client = OpenAI()`)

OpenAI SDK 설치 및 환경변수 설정이 완료되었다면, 이제 **API 요청을 보낼 클라이언트 객체**를 생성할 차례입니다. 클라이언트 객체는 OpenAI의 다양한 API (챗봇, 완성, 편집 등)를 호출하는 기능을 제공합니다. 생성 방법은 매우 간단합니다:

##실습1.5. OpenAI Client Instance 생성 및 확인

```
from openai import OpenAI

# OpenAI 클라이언트 인스턴스 생성
client = OpenAI()
```

위 코드 한 줄로 `client` 인스턴스가 만들어집니다. `OpenAI()` 호출 시 별도의 인자를 넣지 않았는데, 이는 앞서 언급한 대로 **환경 변수에 저장된 API 키**를 자동으로 사용하기 때문입니다.

환경 변수를 사용하지 않고 직접 키를 넣고 싶다면 `client = OpenAI(api_key="sk-...")` 처럼 `api_key` 인자를 지정할 수 있지만, 보안상 노트북에 키를 직접 노출하는 방법은 추천하지 않습니다.

 **확인:** OPENAI_API_KEY 환경변수가 설정되지 않았다면 이 단계에서 오류가 발생할 수 있습니다. 예를 들어 `openai.error.AuthenticationError: No API key provided`와 같은 에러가 뜨는 경우, 이전 단계의 키 등록이 제대로 되었는지 다시 점검하세요. 정상적으로 `client = OpenAI()`가 실행되면 별다른 출력은 없으며, `client` 객체가 메모리에 로드됩니다.

`client` 객체 생성 후 내부 속성을 살펴보고 싶다면 `print(client)` 를 해보세요. OpenAI SDK의 `OpenAI` 클래스 객체임을 확인할 수 있습니다. 이제 이 클라이언트를 통해 모델에게 프롬프트를 보내볼 것입니다.

7. OpenAI API 호출 및

`response.choices[0].message.content` 로 응답 처리 실습

마지막으로, 준비된 `client` 를 사용하여 실제로 OpenAI 모델에 프롬프트를 보내보고 응답을 받아보는 실습을 합니다. 이번 과정에서는 **Chat Completions API**를 사용해 볼 것입니다 (GPT-4o-mini 모델을 기본 사용). **Chat Completions**는 메시지의 배열을 보내 대화형으로 응답을 받는 방식으로, GPT 계열 모델을 가장 유연하게 활용하는 방법입니다.

##실습1.6. OpenAI API 호출 및 테스트

다음 예시는 사용자 메시지를 보내고 모델의 응답을 출력하는 간단한 코드입니다:

```
# 모델에게 보낼 메시지 구성 (역할: user)
message = {"role": "user", "content": "안녕하세요, 오늘 날씨는 어떨까요?"}

# Chat Completions API 호출 (gpt-4o-mini 모델 사용 예)
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[ message ]
)

# 응답에서 assistant의 답변 내용 추출
assistant_reply = response.choices[0].message.content
print("ChatGPT 답변:", assistant_reply)
```

실행하면 **모델의 답변 텍스트**가 `assistant_reply`에 저장되고 출력됩니다. `response` 객체에는 모델 응답의 모든 정보가 들어있는데, 그 중 `choices[0].message.content` 속성이 바로 첫 번째 응답 메시지의 본문 콘텐츠입니다. 따라서 이 값을 사용하여 최종 답변을 얻을 수 있습니다. (`choices` 배열에 여러 항목이 있을 수 있으나, 기본적으로 한 번 요청에 하나의 응답만 받으므로 `[0]` 번째를 사용)

- **응답 객체 구조:** `response`는 OpenAI SDK에서 정의된 **응답 객체**이며, Python의 딕셔너리처럼 다룰 수도 있지만 속성 접근을 권장합니다. 주요 필드로 `response.choices` (응답 목록), `response.usage` (토큰 사용량 정보), `response.created` (응답 생성 시각 UNIX timestamp) 등을 포함합니다. 과거 0.x 버전에서는 `response['choices'][0]['message']['content']` 처럼 딕셔너리 접근을 했으나, 1.x 버전에서는 `response.choices[0].message.content`와 같이 속성 접근법으로 변경되었으니 유의하세요.
- **응답 처리:** 위 코드에서는 단순히 출력만 했지만, 실전에서는 이 `assistant_reply` **변수**를 다음 질문에 활용하거나, 파일로 저장하거나, 후처리를 할 수도 있습니다. 일단 여기서는 제대로 응답이 오는지만 확인하면 됩니다. 예를 들어 사용자 메시지가 "Hello!" 였다면, 모델이 "Hello there! How can I assist you today?"와 같은 응답을 반환할 것입니다.
- **모델 선택:** `model="gpt-4o-mini"`로 지정한 부분을 다른 모델 이름으로 바꾸어 실험해볼 수 있습니다. 예를 들어 `gpt-5`로 변경하면 보다 **고도화된 GPT-5 모델로 응답하지만, 속도나 비용 면에서 차이**가 있습니다. 또는 GPT-4o 계열을 사용하고 싶다면 `model="gpt-4o-mini"`로 지정할 수도 있습니다. **자신의 계정이 해당 모델에 접근 권한이 있는지에 따라** 오류가 날 수도 있으므로, 권한이 없을 경우 사용 가능한 모델을 선택해야 합니다.
- **추가 실습:** 응답 객체의 `usage` 정보를 출력해 보세요. 예: `print(response.usage)`를 하면 `{ "prompt_tokens": ..., "completion_tokens": ..., "total_tokens": ... }` 형태로 이번 대화에 사용된 토큰 수를 알려줍니다. 이 정보는 추후 프롬프트를 최적화하거나 비용을 관리하는 데 유용합니다.

! 문제 해결: 만약 **API 호출에서 에러가 발생한다면, 오류 메시지를 잘 읽고 원인을 파악하세요.**

- `openai.error.AuthenticationError`: **인증 실패**입니다. API 키가 제대로 설정되었는지 확인하세요 (키 오타 또는 미설정 가능).
- `openai.error.InvalidRequestError` 또는 모델 관련 에러: 요청이 잘못되었거나 접근 불가능한 모델을 호출한 경우입니다. **모델 이름이 정확한지, 메시지 포맷이 올바른지** 확인하세요. 권한 문제라면 다른 모델을 쓰거나 **Billing 결제**가 되었는지 점검해야 합니다.

- `AttributeError: 'Message' object is not subscriptable` 등: 응답 처리 시 발생하는 오류로, `response.choices[0].message["content"]` 처럼 잘못된 접근을 한 경우입니다. 객체 속성 접근 방식을 사용해야 합니다 (`.message.content` 형태).
- `Rate limit` 혹은 `Quota exceeded` : 요청 빈도가 너무 높거나 크레딧이 부족한 경우입니다. 잠시 대기했다 재시도하거나 크레딧을 추가하세요.